



TP1: Introducción a programación en Bash

En este tutorial vamos a usar la terminal de Linux para analizar datos genómicos reales. El objetivo es aprender a automatizar tareas mediante el uso de comandos de Bash.

Objetivos

1. Familiarizarse con el uso de la terminal y sus comandos básicos para navegar, crear directorios y manipular archivos.
 2. Obtener y procesar datos biológicos reales desde bases de datos públicas usando herramientas de línea de comandos.
 3. Automatizar tareas utilizando scripts, para procesar múltiples archivos.
-

Uso de programación en bioinformática

En bioinformática, la mayor parte del trabajo consiste en procesar grandes volúmenes de datos. Hacer esto a mano o con programas que tienen ventanas (interfaces gráficas) puede ser lento, tedioso y difícil de repetir.

Aprender a usar la terminal y escribir scripts te permite:

- **Automatizar tareas repetitivas:** Si tenés que procesar 10 archivos, lo hacés con un bucle. Si tenés 1000, el mismo bucle funciona igual. No importa la cantidad.
- **Trabajar con datos masivos:** Los archivos de secuenciación pueden tener cientos de miles de líneas. Abrirlos con un editor de texto es imposible. Desde la terminal podés filtrarlos, buscar patrones y extraer información en segundos.
- **Hacer reproducible tu trabajo:** Un script documenta exactamente qué hiciste con los datos. Si alguien quiere repetir tu análisis, solo tiene

que ejecutar el mismo script. No hay pasos olvidados ni cosas hechas "a mano" que no se puedan replicar.

En este tutorial, vas a experimentar esto en la práctica: vamos a procesar varios genomas completos usando solo comandos y un script.

Proyecto:

Klebsiella pneumoniae es una bacteria que vive normalmente en el intestino de las personas sin causar problemas. Sin embargo, en pacientes internados en hospitales (sobre todo aquellos con sistemas inmunológicos debilitados o con dispositivos médicos como respiradores o catéteres), puede causar infecciones graves como neumonía, infecciones del tracto urinario, bacteriemia (infección en la sangre) y meningitis.

El principal problema con *K. pneumoniae* es su capacidad para volverse resistente a múltiples antibióticos. En las últimas décadas, han aparecido cepas que son resistentes a los **carbapenémicos**, un grupo de antibióticos que se consideran de "último recurso" porque son efectivos contra muchas bacterias multirresistentes.

El gen responsable de esta resistencia en muchas cepas se llama ***blaKPC***. Este gen codifica una enzima (llamada KPC) que destruye a los carbapenémicos, volviéndolos inútiles. El gen se encuentra generalmente en **plásmidos**, que son pequeñas moléculas de ADN que las bacterias pueden intercambiar entre sí, lo que permite que la resistencia se propague rápidamente entre distintas cepas e incluso entre diferentes especies bacterianas.

Los investigadores han identificado que ciertas mutaciones puntuales en el gen *blaKPC* pueden hacer que la enzima sea aún más eficiente, aumentando la capacidad de la enzima para destruir carbapenémicos y, además, puede hacer que sea más difícil de inhibir con nuevos fármacos.

En este tutorial, vamos a recibir los datos de secuenciación del gen *blaKPC* de 5 pacientes de un hospital. Nuestro objetivo será:

1. Analizar las secuencias del gen *blaKPC* de cada paciente.
2. Traducir esas secuencias de ADN a la secuencia proteica.
3. Identificar qué pacientes contienen cepas con la mutación S179G.
4. Entregar un informe al hospital indicando qué pacientes están contagiados con cepas resistentes.

Ejercicio 1: Conocer la proteína de referencia en UniProt

Antes de analizar las secuencias de los pacientes, es importante tener un punto de comparación. Vamos a buscar en UniProt la proteína KPC (codificada por el gen *blaKPC*).

Pasos a seguir:

1. Ingresá a [UniProt](#).
2. En el buscador, escribí "**KPC Klebsiella pneumoniae**" y presioná Enter.
3. Buscá la entrada *****Q9F663***** que corresponda a la variante **KPC-2**.
4. Hacé clic en la entrada para ver los detalles.
5. Tomá nota de la **longitud** de la proteína (aparece en la parte superior, justo debajo del nombre).
6. Andá a la sección "**Phenotypes & Variants**" y buscá la subsección "**Mutagenesis**". Observá la tabla de mutagénesis.

⇒ Preguntas:

a) ¿Cuántas mutaciones están documentadas en la tabla? ¿En qué posiciones de la proteína ocurren?

b) Identificá las **dos mutaciones** que, según la descripción, **aumentan la eficiencia catalítica de la enzima y generan resistencia a ceftazidima**.

Anotá:

- La posición de cada mutación.
- El cambio de aminoácido que producen (ej: "P → R").
- El efecto que tienen sobre la enzima (leé la descripción completa).

c) Según la tabla, ¿estas dos mutaciones actúan de forma independiente o **necesitan estar juntas** para generar el efecto? ¿Qué frase de la tabla te

indica eso?

Nota clave: Prestá atención a la frase "**when associated with**" (cuando está asociada con). Esto indica que las mutaciones en las posiciones 103 y 239 **no actúan solas**: una cepa necesita tener **ambas mutaciones** para presentar el aumento de eficiencia contra ceftazidima que se describe en la tabla.

Hasta acá, descubrimos varias cosas importantes:

1. La proteína KPC-2 tiene una longitud de 293 aa.
2. En la tabla de mutagénesis de UniProt hay mutaciones documentadas en tres posiciones: 103, 178 y 239.
3. Las mutaciones en las posiciones **103** (cambio de P a R) y **239** (cambio de G a R, probablemente) **no generan resistencia por sí solas**. Para que la enzima aumente su eficiencia contra ceftazidima ~40 veces, **las dos mutaciones tienen que estar presentes al mismo tiempo** en la misma cepa.
4. Esto significa que, cuando analicemos las secuencias de los pacientes, no basta con buscar una sola mutación. Vamos a tener que verificar si cada paciente tiene **ambas mutaciones** para determinar si realmente presenta el fenotipo de alta resistencia.

Ejercicio 2: Preparar el entorno de trabajo

Introducción a Linux / Ubuntu

Linux es un sistema operativo de código abierto, estable y muy utilizado en entornos de investigación y servidores.

Ubuntu es una de sus distribuciones más populares, conocida por su facilidad de uso y amplia comunidad de soporte.

En bioinformática y análisis de datos, Linux ofrece ventajas importantes: permite manejar grandes volúmenes de información, automatizar tareas mediante scripts, instalar software científico fácilmente y ejecutar pipelines de análisis de manera reproducible.

A través de la terminal, los usuarios pueden interactuar directamente con el sistema operativo, ejecutar programas, gestionar archivos y carpetas, y combinar herramientas de manera flexible.

Introducción a Bash

Bash (Bourne Again SHell) es un intérprete de comandos que permite interactuar con el sistema operativo mediante texto.

Usando Bash, los usuarios pueden navegar entre carpetas, gestionar archivos, ejecutar programas, combinar comandos y automatizar tareas repetitivas mediante scripts.

En bioinformática, Bash es fundamental para procesar grandes volúmenes de datos, lanzar pipelines de análisis y manipular archivos de secuenciación de manera eficiente sin necesidad de interfaces gráficas.

Introducción al uso de la terminal

En Windows (con WSL)

- **Método rápido:** Abrí el menú de Inicio, escribí "**Ubuntu**" (o el nombre de la distribución que instalaste) o WSL y hacé clic para abrirla.
- **Desde la línea de comandos de Windows:** Si ya tenés una terminal de Windows (CMD o PowerShell) abierta, escribí `ws1` y presioná Enter. Esto te lleva directamente a la terminal de Linux.

Verificación: Cuando veas algo como `usuario@nombre_pc:~$`, significa que la terminal está lista para recibir comandos.

Nota: Si estás en **Linux o macOS**, ya tenés una terminal nativa. Solo buscá "Terminal" en el menú de aplicaciones o usá el atajo de teclado correspondiente (Ctrl+Alt+T en Linux, o Comando+Espacio en macOS).

Se abrirá una ventana de terminal lista para usar!

¿Qué es un comando?

Conceptos básicos al escribir comandos

- Los comandos se escriben en **minúsculas** y respetando espacios entre palabras.
- Un comando suele tener la siguiente estructura:

```
comando [opciones] [argumentos]
```

- **comando:** lo que queremos ejecutar (por ejemplo `ls`, `cd`, `wc`).
- **opciones:** modifican el comportamiento del comando (por ejemplo `-l` para listar en detalle).
- **argumentos:** archivos o carpetas sobre los que actúa el comando.
- La terminal es **sensible a mayúsculas y minúsculas**, por lo que `File.txt` y `file.txt` son archivos distintos.
- Se pueden usar **tabulaciones** para autocompletar nombres de archivos

o carpetas, evitando errores de tipeo.

- Los **comentarios** se escriben con `#` y la terminal los ignora, útil para documentar scripts.

Comandos básicos de Bash

A continuación hay una lista de los comandos más usados en Bash, vamos a ir viendo los más importantes para este curso en la siguiente sección

Comando	Qué hace	Ejemplo
<code>pwd</code>	Muestra la ruta del directorio actual	<code>pwd</code> → <code>/home/usuario/proyecto</code>
<code>ls</code>	Lista los archivos y carpetas del directorio	<code>ls</code> → <code>archivo1.txt archivo2.txt</code>
<code>cd</code>	Cambia de directorio	<code>cd datos/</code> → cambia al subdirectorio <code>datos</code>
<code>mkdir</code>	Crea un nuevo directorio	<code>mkdir resultados</code> → crea carpeta <code>resultados</code>
<code>rmdir</code>	Elimina directorios vacíos	<code>rmdir carpeta_vacia</code> → borra la carpeta
<code>rm</code>	Elimina archivos	<code>rm archivo.txt</code> → borra <code>archivo.txt</code>
<code>cp</code>	Copia archivos	<code>cp archivo.txt backup/</code> → copia <code>archivo</code> a <code>backup/</code>
<code>mv</code>	Mueve o renombra archivos	<code>mv archivo.txt archivo_old.txt</code> → renombra <code>archivo</code>
<code>head</code>	Muestra las primeras líneas de un archivo	<code>head sample.fastq</code> → primeras 10 líneas
<code>tail</code>	Muestra las últimas	<code>tail sample.fastq</code> → últimas 10

Comando	Qué hace	Ejemplo
	líneas de un archivo	líneas
<code>wc</code>	Cuenta líneas, palabras o caracteres	<code>wc -l sample.fastq</code> → número de líneas
<code>cat</code>	Muestra el contenido de un archivo	<code>cat archivo.txt</code> → imprime el archivo en pantalla
<code>less</code>	Permite ver un archivo página por página	<code>less archivo.txt</code> → navegación interactiva
<code>grep</code>	Busca texto dentro de un archivo	<code>grep "ATG" sample.fasta</code> → muestra líneas con "ATG"
<code>chmod</code>	Cambia permisos de archivo o carpeta	<code>chmod 755 script.sh</code> → permisos de ejecución
<code>echo</code>	Imprime texto en la terminal.	<code>echo "Hola mundo"</code> → imprime "Hola mundo" en la terminal
<code>wget</code>	Descarga archivos desde una URL	<code>wget https://example.org/file.fasta</code> → descarga el archivo
<code>awk</code>	Procesa texto y columnas de archivos	<code>awk '{print \$1}' archivo.tsv</code> → imprime la primera columna
<code>gunzip</code>	Descomprime archivos <code>.gz</code>	<code>gunzip archivo.fasta.gz</code> → descomprime el archivo
<code>zcat</code>	Muestra el contenido de un archivo <code>.gz</code> sin descomprimir	<code>zcat archivo.fasta.gz</code> → imprime el archivo en pantalla
<code>./</code>	Ejecuta un script o	<code>./script.sh</code> → ejecuta <code>script.sh</code>

Comando	Qué hace	Ejemplo
	programa en el directorio actual	
,	,	Pipe: envía la salida de un comando como entrada de otro
*	Comodín que representa cualquier cadena de caracteres	<code>ls *.fastq</code> → lista todos los archivos que terminan en <code>.fastq</code>

Errores más comunes en Bash

Tipo de error	Mensaje típico	Descripción
Comando no encontrado	<code>command not found</code>	El comando está mal escrito o no está instalado en el sistema.
Archivo o directorio inexistente	<code>No such file or directory</code>	La ruta es incorrecta, el archivo no existe o no estás en el directorio adecuado.
Permiso denegado	<code>Permission denied</code>	Se intenta acceder o ejecutar un recurso sin permisos suficientes.
Error de sintaxis	<code>syntax error</code>	Uso incorrecto de paréntesis, comillas, corchetes u otros símbolos.
Opción inválida	<code>invalid option</code>	Se usó una bandera que el comando no reconoce.

Tipo de error	Mensaje típico	Descripción
Argumentos faltantes	—	El comando requiere argumentos adicionales para poder ejecutarse correctamente.
Archivo en uso por otro proceso	—	El archivo está siendo utilizado por otro programa y no puede modificarse.
Variables o rutas mal expandidas	—	Errores en el uso de variables (<code>\$var</code>), <code>~</code> o comillas.
Sobrescritura accidental o salida vacía	—	El comando se ejecuta sin error, pero produce un resultado incorrecto o vacío.
Espacios en nombres de archivos	—	Rutas con espacios requieren comillas o escape con <code>\</code> .
Entorno mal configurado	—	Problemas con <code>PATH</code> , entornos virtuales o módulos no cargados.
Redirecciones o pipes incorrectos	—	Uso incorrecto de <code>`</code>
Bucles infinitos en scripts	—	Errores lógicos que impiden que el script termine.

Tipo de error	Mensaje típico	Descripción
Caracteres especiales o codificación	—	Caracteres no visibles, tildes o emojis pueden romper comandos.

Navegación y cambio de directorio en Bash

En la terminal de Linux/Ubuntu, es fundamental saber **dónde estamos ubicados** y cómo movernos entre carpetas.

Esto permite ejecutar comandos sobre los archivos correctos y organizar proyectos de manera eficiente.

Comando para ver la ubicación actual

```
pwd
```

- `pwd` (print working directory) muestra la ruta completa del directorio actual.
- Saber en qué carpeta estamos evita errores al ejecutar comandos que modifican archivos.

🔗 Pregunta 1

Abrir una terminal y ejecutar el comando para obtener la ubicación actual

```
pwd
```

Estructura de directorios de Ubuntu

La organización de archivos en Ubuntu y Lubuntu es bastante diferente a la de

Windows. Si bien no vamos a detallar completamente toda la estructura y que es cada carpeta (porque el 95% no lo van a usar en esta materia), es importante tener una idea de lo básico:

- / Carpeta raíz, o *root*. Contiene al resto de las carpetas
 - **/etc**: Configuraciones del sistema para todos los usuarios (mucho cuidado al tocar)
 - **/home**: Ubicación de los directorios de los diferentes usuarios (o en este caso el único usuario)
 - **/home/user**: Directorio del usuario *user*. Es el lugar donde van a trabajar la mayoría del tiempo (incluye tanto el Escritorio como Documentos) y donde se abre por defecto la terminal (más sobre esto en un ratito). Comúnmente referida como *home directory* o *home* del usuario *user*
 - **/media**: Si fuera una computadora normal (no VM) aca aparecerían los pendrives. En nuestro caso aca aparecen por defecto las carpetas compartidas con la PC host
 - **/tmp**: Ubicación de los archivos temporales de los programas
 - **/var**: Ubicación de los archivos variables de los programas, como logs, bases de datos, paginas webs, etc
 - **/var/log**: Probablemente la subcarpeta más usada de **/var**. Contiene los logs de los programas (que a veces es la única forma de saber porque algo no anduvo)

Esto es simplemente un vistazo rápido. Si quieren la lista completa de subdirectorios de Ubuntu la pueden encontrar en [esta página](#), pero tengan en cuenta que tocar cualquier cosa fuera de **/home** conlleva la posibilidad de arruinar la computadora. En esta materia vamos a usar principalmente **/home** y **/media**.

Paths relativos y absolutos

En programación, un path (o ruta) es la cadena de caracteres que indica la ubicación de un archivo o directorio dentro del sistema de archivos.

Paths absolutos

Un **path absoluto** es aquel que especifica la ubicación completa y exacta de un archivo o directorio desde la raíz del sistema. Esto significa que no depende de dónde estés parado actualmente; siempre apunta al mismo lugar sin importar el directorio de trabajo. Por ejemplo, en Linux o macOS, un path absoluto comienza con una barra inclinada (/), como en

`/home/usuario/documentos/informe.txt` . En Windows, comienza con la letra de la unidad, como en `C:\Users\Usuario\Documentos\informe.txt` . Los paths absolutos son útiles cuando necesitas referenciar un archivo desde cualquier parte del sistema sin ambigüedades.

Paths relativos

Un **path relativo**, en cambio, indica la ubicación de un archivo o carpeta en relación al directorio actual de trabajo (el lugar donde estás parado en la terminal o desde donde se ejecuta el programa). No parte desde la raíz, sino desde la posición actual. Por ejemplo, si estás en `/home/usuario` y querés acceder a `documentos/informe.txt` , el path relativo sería `documentos/informe.txt` (si la carpeta está dentro del directorio actual) o `../documentos/informe.txt` si está un nivel arriba. Los paths relativos son más cortos y flexibles, ya que permiten mover proyectos completos de una carpeta a otra sin tener que reescribir todas las rutas.

Ambos tipos de paths tienen sus ventajas y desventajas.

Los **paths absolutos** tienen la ventaja de funcionar siempre, pero al usar toda la estructura toman más tiempo de escribir y son más susceptibles a cambios de directorios (si muevo un archivo de lugar tengo que reescribir el comando).

Por otro lado, los **paths relativos** son mucho más rápidos de escribir y en muchos casos funcionan en diferentes ubicaciones (o computadoras), pero al depender de la ubicación de la terminal esto puede causar problemas si pienso que estoy en una carpeta pero estoy realmente en otra. En esta cursada vamos a usar ambos para diferentes casos.

Además de estos dos tipos, es importante conocer algunos símbolos especiales que se usan en los paths:

- El punto (.) representa el directorio actual.
- Los dos puntos (..) representan el directorio padre (un nivel arriba).
- La barra inclinada (/) se usa en Linux, macOS y sistemas Unix como separador de carpetas.
- La barra invertida (\) se usa en Windows como separador (aunque muchas herramientas modernas aceptan también la barra inclinada).

Entender la diferencia entre paths absolutos y relativos es clave para evitar errores como "archivo no encontrado", especialmente cuando trabajás en proyectos que se mueven entre distintos equipos o sistemas operativos.

Crear y eliminar carpetas

En el sistema de archivos de Linux, las carpetas (también llamadas directorios) son la forma principal de organizar archivos. Para trabajar de forma ordenada en bioinformática, es fundamental saber crear y eliminar carpetas desde la terminal.

Los comandos `mkdir` y `rmdir` son las herramientas básicas para gestionar directorios:

- `mkdir` → Crear nuevas carpetas.
- `rmdir` → Eliminar carpetas vacías.

Sintaxis general

```
# Crear una carpeta
mkdir [opciones] carpeta

# Eliminar una carpeta vacía
rmdir [opciones] carpeta
```

- `carpeta` → nombre de la carpeta a crear o eliminar
- `[opciones]` → parámetros adicionales como `-p` para crear carpetas anidadas

Ejemplos

Los ejemplos que aparecen a continuación sirven solo para entender la sintaxis, no los corran!

El código que deben ejecutar está indicado en la sección Ejercicios.

Crear una carpeta simple

```
mkdir resultados
```

Crear subcarpetas de manera recursiva

```
mkdir -p datos/fastq
```

Eliminar una carpeta vacía

```
rmdir carpeta_vacia
```

Eliminar una carpeta con contenido (¡cuidado!)

```
rm -r carpeta_con_datos
```

Buenas prácticas

- Revisar siempre el contenido antes de eliminar carpetas con `rm -r .`
- Mantener una estructura clara de proyecto, por ejemplo:

```
project/  
├─ datos/  
├─ scripts/  
└─ results/
```

- Usar nombres descriptivos para carpetas para facilitar la organización de los análisis.

🔗 **Pregunta 2**

Completar y ejecutar el comando para generar una carpeta de "TP01"

```
mkdir
```

Cambiar de directorio

```
cd nombre_de_carpeta
```

- `cd` (change directory) cambia la ubicación actual a la carpeta indicada.
- Se puede usar una **ruta relativa** (`cd subcarpeta`) o **ruta absoluta** (`cd /home/usuario/proyecto`).
- Para subir un nivel en la jerarquía de carpetas se usa: `cd ..`
- Para ir al directorio personal del usuario se usa: `cd ~`

Ejemplos

Los ejemplos que aparecen a continuación sirven solo para entender la sintaxis, no los corran!

El código que deben ejecutar está indicado en la sección Ejercicios.

```
cd datos/                # va al subdirectorio "datos"
cd ../                  # sube un nivel en la jerarquía de carpetas
cd /home/usuario/proyecto # va directamente a la ruta absoluta
```

Errores más comunes en para `cd`

Error	Ejemplo en terminal	Mensaje de error
Directorio no existe	<code>\$ cd datos</code>	<code>bash: cd: datos: No such file or directory</code>
Ruta mal escrita (mayúsculas/ minúsculas)	<code>\$ cd Documentos</code>	<code>bash: cd: Documentos: No such file or directory</code>
Espacios en nombres de carpetas	<code>\$ cd Mi Carpeta</code>	<code>bash: cd: Mi: No such file or directory</code>
Permiso denegado	<code>\$ cd /root</code>	<code>bash: cd: /root: Permission denied</code>

Error	Ejemplo en terminal	Mensaje de error
Ruta relativa inexistente	<code>\$ cd ../datos</code>	<code>bash: cd: ../datos: No such file or directory</code>
Intentar entrar a un archivo	<code>\$ cd archivo.txt</code>	<code>bash: cd: archivo.txt: Not a directory</code>

Buenas prácticas

1. Siempre verificar la ubicación actual con `pwd` antes de cambiar de carpeta.
2. Usar **tabulaciones** para autocompletar nombres de carpetas y evitar errores de tipeo.
3. Documentar scripts indicando rutas relativas o absolutas para que sean reproducibles.

🔗 Pregunta 3

Ejecutar el comando para ingresar a la carpeta "TP01".

```
cd TP01
```

🔗 Pregunta 4

Dentro de la carpeta "TP01" generar las subcarpetas necesarias para obtener la siguiente estructura de datos:

```
TP01/  
├─ datos/  
├─ scripts/  
└─ resultados/
```

Listar contenido de carpetas

El comando `ls` es uno de los más utilizados en la terminal. Permite **listar el contenido de un directorio**, mostrando los archivos y subcarpetas que contiene. Es la herramienta fundamental para explorar el sistema de archivos y verificar qué hay en cada carpeta.

Sintaxis general

```
ls [opciones] [ruta]
```

- `ruta` → directorio a listar (si no se especifica, lista el directorio actual).
- `[opciones]` → modificadores que cambian el formato o la información que se muestra.

Opciones más comunes

Opción	¿Qué hace?
<code>-l</code>	Muestra información detallada (permisos, dueño, tamaño, fecha).
<code>-a</code>	Muestra todos los archivos, incluyendo los ocultos (los que empiezan con <code>.</code>).
<code>-h</code>	Muestra los tamaños en formato legible (KB, MB, GB) – debe usarse con <code>-l</code> .

Opción	¿Qué hace?
<code>-lh</code>	Combina <code>-l</code> y <code>-h</code> : detalles con tamaños legibles.
<code>-R</code>	Lista de forma recursiva (todas las subcarpetas).
<code>-S</code>	Ordena por tamaño (más grande primero).
<code>-t</code>	Ordena por fecha de modificación (más reciente primero).

Ejemplos

Los ejemplos que aparecen a continuación sirven solo para entender la sintaxis, **¡no los corran!**

El código que deben ejecutar está indicado en la sección Ejercicios.

Listar el contenido del directorio actual

```
ls
```

Listar con detalles y tamaños legibles

```
ls -lh
```

Listar **todos** los archivos (incluyendo ocultos) con detalles

```
ls -la
```

Listar una carpeta específica

```
ls -lh /home/estudiante/documentos/
```

Ordenar por tamaño con detalles

`ls -ls`

Errores comunes

- `ls carpeta_inexistente` → Si la carpeta no existe, aparece el error `cannot access 'carpeta_inexistente': No such file or directory`.
- `ls -l archivo_inexistente` → Error similar: `cannot access 'archivo_inexistente': No such file or directory`.
- Olvidar la opción `-a` y no ver archivos ocultos (como `.bashrc`, `.config`, etc.).

Buenas prácticas

- Usar `ls -lh` en lugar de solo `ls` para ver tamaños de archivos de forma clara.
- Combinar `-l` y `-a` con `-h` para tener una visión completa del directorio: `ls -lah`.
- Si estás explorando una carpeta con muchos archivos, podés usar `ls | head -n 10` (todavía no vimos `head`, pero lo usaremos más adelante) para ver solo los primeros 10 resultados.
- No confiar en `ls` solo para ver si un archivo existe: combinarlo con `ls -l archivo_especifico` o usar `file` para ver el tipo de contenido.

➡ Pregunta 5

Dentro de la carpeta TP01 ejecutar `ls -lh` para verificar que contenga la estructura recomendada.

TP01/

├─ datos/

├─ scripts/

└─ resultados/

Ejercicio 3: Descargar y explorar los datos

En este ejercicio vamos a trabajar con el archivo que contiene las secuencias de los pacientes. El hospital nos ha proporcionado un repositorio interno donde están almacenados los datos de secuenciación. Vamos a descargar el archivo desde ese repositorio, explorar su contenido para entender su estructura y extraer información clave.

Paso 1: Descargar el archivo

`wget` es un comando que permite descargar archivos desde internet o desde servidores internos directamente desde la terminal. Es muy útil porque funciona en segundo plano, puede reanudar descargas interrumpidas y no necesita un navegador web.

Sintaxis general

```
wget [opciones] URL
```

- `URL` → dirección del archivo que queremos descargar.
- `[opciones]` → modificadores que cambian el comportamiento de la descarga.

Opciones más comunes

Opción	¿Qué hace?
<code>-O nombre</code>	Guarda el archivo con el nombre que le indiquemos (si no se usa, se guarda con el nombre original de la URL).
<code>-q</code>	Modo silencioso (no muestra mensajes en pantalla).
<code>-c</code>	Reanuda una descarga interrumpida.
<code>-i archivo</code>	Lee una lista de URLs desde un archivo y descarga todas.

Ejemplos

Los ejemplos que aparecen a continuación sirven solo para entender la sintaxis, ¡no los corran! El código que deben ejecutar está indicado en la sección Ejercicios.

Descargar un archivo y guardarlo con el nombre original:

```
wget "https://ejemplo.com/archivo.txt"
```

Descargar un archivo y guardarlo con otro nombre:

```
wget -O nuevo_nombre.txt "https://ejemplo.com/archivo.txt"
```

Errores comunes

- `wget: unable to resolve host address` → La URL está mal escrita o no hay conexión a internet.
- `wget: HTTP error 404` → El archivo no existe en el servidor.
- `wget: error de permiso` → No tenés permisos para escribir en la carpeta donde estás guardando.

Buenas prácticas

- Usar `-O` para tener control sobre el nombre del archivo y evitar confusiones.
- Verificar la URL antes de ejecutar `wget` para evitar errores 404.
- Si descargas archivos grandes, usar `-c` para poder reanudar la descarga si se corta.

➡ Pregunta 6

Dentro de la carpeta del TP01 ingresar a la carpeta `datos/`. Ejecutar el siguiente comando para obtener las secuencias:

```
wget https://raw.githubusercontent.com/mercedesgarnham/Tutoriales/refs/heads/main/T
```

🔗 Pregunta 7

Verificar que el archivo se haya descargado correctamente usando `ls`

```
ls
```

Manuales en Linux

En Linux, los **manuales** (o páginas `man`) son la documentación oficial integrada en el sistema para casi todos los comandos, programas y utilidades disponibles. Son la fuente más confiable y completa para entender cómo funciona una herramienta, qué opciones tiene y cómo usarla correctamente.

Cómo usar el comando `man`

Para acceder al manual de un comando, simplemente escribís `man` seguido del nombre del comando:

```
man wget
```

Esto abre la página del manual de `wget` con su sintaxis, descripción de opciones, ejemplos y más.

Estructura de una página de manual

Las páginas de manual suelen tener esta estructura estándar:

- **NAME:** nombre del comando y una breve descripción de lo que hace.
- **SYNOPSIS:** muestra la sintaxis básica del comando, indicando qué argumentos y opciones puede recibir.

- **DESCRIPTION:** explica en detalle qué hace el comando y cómo funciona.
- **OPTIONS:** lista todas las opciones o banderas (flags) que acepta el comando, con su descripción.
- **EXAMPLES:** ejemplos prácticos de uso (no siempre está presente, pero es muy común).
- **SEE ALSO:** referencias a otros comandos o manuales relacionados.

Navegación dentro del manual

Una vez que abriste el manual con `man`, podés navegar de la siguiente forma:

- **Flechas / Page Up / Page Down:** moverse arriba y abajo.
- **/palabra:** buscar una palabra dentro del manual (presioná `n` para ir a la siguiente coincidencia).
- **q:** salir del manual y volver a la terminal.

Paso 2: Visualizar el archivo

Los comandos `head` y `tail` permiten **ver partes de un archivo de texto** sin abrirlo completo, lo cual es útil para inspeccionar archivos grandes.

**** Ver el comienzo de un archivo con `head` ****

- Muestra las primeras líneas de un archivo.
- Por defecto, muestra las **primeras 10 líneas**, pero se puede cambiar con la opción `-n`.

Ejemplos

Los ejemplos que aparecen a continuación sirven solo para entender la sintaxis, no los corran! El código que deben ejecutar está indicado en la sección Ejercicios.

```
# Ver las primeras 10 líneas de sample.fastq
head sample.fastq
```

```
# Ver las primeras 20 líneas
head -n 20 sample.fastq
```

**** Ver el final de un archivo con `tail` ****

- Muestra las últimas líneas de un archivo.
- También por defecto son 10 líneas, modificables con `-n`.

```
# Ver las últimas 10 líneas de sample.fastq
tail sample.fastq
```

```
# Ver las últimas 15 líneas
tail -n 15 sample.fastq
```

Errores más comunes en para `head` y `tail`

Error	Ejemplo en terminal	Mensaje típico	Explicación
Archivo no existe	<code>\$ head datos.txt</code>	<code>No such file or directory</code>	El archivo no está presente o no existe en la ruta especificada.
Opción inválida	<code>\$ head -z archivo.txt</code>	<code>invalid option -- 'z'</code>	Se usó una opción no reconocida por el comando.
Valor no válido para <code>-n</code>	<code>\$ head -n diez archivo.txt</code>	<code>invalid number of lines</code>	La opción <code>-n</code> requiere un número entero, no una palabra.
Archivos	<code>\$ head archivo_muy_grande.fastq</code>	—	En archivos muy grandes, puede ser necesario usar <code>head -n</code> para limitar el número de líneas mostradas.

Error	Ejemplo en terminal	Mensaje típico	Explicación
muy grandes			lectura de disco parece que se cubren
Falta de permisos de lectura	<code>\$ tail /root/secret.log</code>	<code>Permission denied</code>	El usuario no tiene permisos de lectura en el archivo
Intentar leer un directorio	<code>\$ head carpeta/</code>	<code>Is a directory</code>	head no funciona sobre directorios
Archivo comprimido	<code>\$ head datos.csv.gz</code>	salida ilegible	El archivo está comprimido con gzip
Flag incompleto	<code>\$ tail -n archivo.txt</code>	<code>option requires an argument</code>	Falta de límite de líneas
Rutas con espacios	<code>\$ head Mis Datos/datos.txt</code>	<code>No such file or directory</code>	Usar comillas o escapar espacios: <code>"Mis Datos/datos.txt"</code> o <code>Mis\Datos\datos.txt</code>

Buenas prácticas

1. Usar `head` para revisar rápidamente la estructura de archivos grandes.
2. Usar `tail` para monitorear archivos que se actualizan continuamente (como logs).

➡ Pregunta 8

Visualiza el archivo ejecutando el siguiente comando:

```
head pacientes.fasta
```

Paso 3: Buscar los nombres de los pacientes con grep

`grep` (Global Regular Expression Print) es un comando que busca líneas que coincidan con un patrón (expresión regular) dentro de uno o varios archivos. Es una de las herramientas más poderosas para filtrar y extraer información de archivos de texto.

Sintaxis general

```
grep [opciones] "patrón" archivo(s)
```

- `"patrón"` → la expresión que queremos buscar.
- `archivo(s)` → el archivo o archivos donde buscar.
- `[opciones]` → modificadores que cambian el comportamiento.

Opciones más comunes

Opción	¿Qué hace?
<code>-i</code>	Ignora mayúsculas/minúsculas.
<code>-v</code>	Muestra las líneas que no coinciden con el patrón.
<code>-c</code>	Cuenta cuántas líneas coinciden (en lugar de mostrarlas).
<code>-l</code>	Muestra solo los nombres de los archivos que contienen el patrón.

Opción	¿Qué hace?
-n	Muestra el número de línea de cada coincidencia.
-w	Busca la palabra exacta (no partes de palabras).
-E	Permite usar expresiones regulares extendidas (ej: `

Ejemplos

Los ejemplos que aparecen a continuación sirven solo para entender la sintaxis, **¡no los corran!** El código que deben ejecutar está indicado en la sección Ejercicios.

Buscar líneas que contengan la palabra "Paciente" en un archivo:

```
grep "Paciente" archivo.txt
```

Buscar líneas que empiecen con el símbolo > en un archivo FASTA:

```
grep "^>" secuencias.fasta
```

Buscar líneas que **no** contengan la palabra "mutación":

```
grep -v "mutación" archivo.txt
```

Contar cuántas líneas tienen "ERR" en un archivo de log:

```
grep -c "ERR" log.txt
```

Buscar líneas que contengan "TP53" o "BRCA1":

```
grep -E "TP53|BRCA1" genes.txt
```

Errores comunes

- `grep: archivo: No such file or directory` → El archivo no existe o escribiste mal el nombre.
- `grep: Unmatched ( or` → Error en la expresión regular (falta cerrar un paréntesis, etc.).
- El patrón no devuelve nada aunque sabés que debería → revisá mayúsculas/minúsculas (usá `-i`) o el símbolo de inicio/fin (`^` y `$`).

Buenas prácticas

- Usar `"^"` para buscar líneas que **empiecen** con algo.
- Usar `"$"` para buscar líneas que **terminen** con algo.
- Encerrar el patrón entre comillas para evitar que el shell interprete caracteres especiales.
- Combinar `grep` con otros comandos mediante tuberías (`|`) para filtrados más complejos.
- Siempre verificar que el archivo existe con `ls` antes de usar `grep`.

⇒ Pregunta 9

Buscá todas las líneas que empiezan con `>` (el símbolo de cabecera en FASTA) en el archivo:

```
grep "^>" pacientes.fasta
```

Paso 4: Guardar los nombres en una tabla

En la terminal, la mayoría de los comandos muestran su salida en la pantalla (stdout). El símbolo `>` permite **redirigir** esa salida a un archivo, en lugar de mostrarla en la terminal. Es útil para guardar resultados, generar archivos de

registro o crear listados que vamos a usar más adelante.

Sintaxis general

```
comando > archivo_salida
```

- `comando` → cualquier comando que genere salida (ej: `grep`, `ls`, `head`).
- `archivo_salida` → el nombre del archivo donde queremos guardar la salida.

Opciones

Símbolo	¿Qué hace?
<code>></code>	Redirige la salida a un archivo y sobrescribe el archivo si ya existe.
<code>>></code>	Redirige la salida a un archivo y agrega al final (no sobrescribe).
<code>2></code>	Redirige los mensajes de error (stderr) a un archivo.
<code>`</code>	<code>`</code> (tubería)

Ejemplos

Los ejemplos que aparecen a continuación sirven solo para entender la sintaxis, **¡no los corran!** El código que deben ejecutar está indicado en la sección Ejercicios.

Guardar la lista de archivos de una carpeta en un archivo:

```
ls -l > lista_archivos.txt
```

Guardar el resultado de una búsqueda con `grep` en un archivo:

```
grep "Paciente" pacientes.fasta > pacientes_encontrados.txt
```

Agregar más resultados al mismo archivo sin borrar lo anterior:

```
grep "Paciente_00" pacientes.fasta >> pacientes_encontrados.txt
```

Guardar los mensajes de error en un archivo separado:

```
wget -O archivo.txt "URL_invalida" 2> errores.txt
```

Errores comunes

- `archivo: Permission denied` → No tenés permiso para escribir en esa carpeta.
- Olvidar que `>` sobrescribe, y perder información importante.
- Usar `>` con un comando que no produce salida (el archivo queda vacío).

Buenas prácticas

- Usar `>>` cuando quieras agregar información a un archivo existente.
- Verificar el contenido del archivo generado con `cat` o `head` para confirmar que la redirección funcionó.
- Usar nombres de archivo descriptivos (ej: `pacientes_nombres.csv` en lugar de `salida.txt`).
- Si vas a sobrescribir un archivo importante, considerá hacer una copia de seguridad primero.

🔑 Pregunta 10

Guardá la lista de nombres de los pacientes (las líneas que comienzan con `>`)

en un archivo dentro de la carpeta `resultados/` :

```
grep "^>" pacientes.fasta > ../resultados/pacientes_nombres.csv
```

📌 **Pregunta 11**

Revisa que el archivo `pacientes_nombres.csv` se haya generado correctamente

Ejercicio 4: Generar el informe final

En este ejercicio vamos a trabajar con las secuencias de ADN de los pacientes para convertirlas en proteínas y así poder identificar las mutaciones en las posiciones 103 y 239.

Lo que vamos a hacer:

1. Extraer el aminoácido en la posición 103 de cada paciente.
2. Extraer el aminoácido en la posición 239 de cada paciente.
3. Determinar qué pacientes tienen las mutaciones que confieren resistencia.

Paso 0: Diagramar el script

Vamos a pensar que pasos son necesarios llevar a cabo para realizar este análisis.

Paso 1: Descargar el script

El script ya está disponible en el repositorio del curso. Ingresá a la carpeta `scripts/` y ejecuta lo siguiente:

```
wget https://raw.githubusercontent.com/mercedesgarnham/Tutoriales/refs/heads/main/TP1
```

Paso 2: Dar permisos de ejecución al script

Para poder ejecutar el script, necesita permisos de ejecución. Usá el comando `chmod` :

```
chmod +x ~/tp_resistencia/scripts/analisis_resistencia.sh
```

Verificación: Ejecutá `ls -l ~/tp_resistencia/scripts/analisis_resistencia.sh` y verificá que los permisos incluyan `-rwxr-xr-x` (la `x` indica que es ejecutable).

Paso 3: Ejecutar el script

Ahora ejecutá el script desde la terminal:

```
bash analisis_resistencia.sh
```

Salida esperada: El script va a guardar un informe en `resultados/informe_final.tsv`.

Verificación: Abrí el archivo generado para confirmar que se creó correctamente:

```
head ../resultados/informe_final.tsv
```

Paso 4: Completar la tabla

Usando los datos que obtuvo el script, completá la siguiente tabla:

Paciente	Longitud (aa)	Pos103	Pos239
Referencia (canónica)	293	P	V
Resistente	293	R	G
Paciente_001			
Paciente_002			
Paciente_003			
Paciente_004			
Paciente_005			

➤ **Pregunta 12**

¿Qué paciente/s está/n infectados con una cepa resistente?

Cierre:

1. Que tipos de experimentos hicimos hoy?
2. Completá la tabla con los temas vistos hoy:

Tema	Algoritmo	Tipo de datos	Base de datos	Análisis del resultado
(completar)	(completar)	(completar)	(completar)	(completar)

3. Cumpliste con los objetivos del tutorial?

Objetivo	Se cumplió?
1. Familiarizarse con el uso de la terminal y sus comandos básicos para navegar, crear directorios y manipular archivos.	Si / No
2. Obtener y procesar datos biológicos reales desde bases de datos públicas usando herramientas de línea de comandos.	Si / No
3. Automatizar tareas utilizando scripts, para procesar múltiples archivos.	Si / No